부록 · LangChain4j · Spring AI

Spring Boot Java + Kotlin 실전 예제

XRP 60분봉 200개 → MA5 · MA20 · RSI14 → LangChain4j → Ollama Gemma3

```yaml
Ollama: http://localhost:11434
```

1. 전체 흐름

```text
업비트 KRW-XRP 60분봉 200개
        ↓
종가(trade_price) 추출
        ↓
MA5 / MA20 / RSI14 계산
        ↓
LangChain4j
        ↓
Ollama  http://localhost:11434
        ↓
Gemma3
        ↓
상승 / 하락 / 횡보 분석
        ↓
Spring Boot REST API
```

2. 프로젝트 구조

```text
upbit-ai/
├─ config/
│  └─ OllamaConfig
├─ controller/
│  └─ XrpController
├─ dto/
│  ├─ CandleDto
│  └─ XrpAnalysisDto
├─ service/
│  ├─ UpbitService
│  ├─ IndicatorService
│  └─ XrpAiService
└─ UpbitAiApplication
```

3. 클래스 역할 비교

클래스	역할
CandleDto	업비트 캔들 JSON을 객체로 받음
UpbitService	KRW-XRP 60분봉 200개 조회
IndicatorService	MA5, MA20, RSI14 계산
XrpAiService	계산 결과를 Gemma3에 전달

클래스	역할
XrpController	/api/xrp/predict REST API 제공

4. application.properties

```properties
server.port=8080

ollama.base-url=http://localhost:11434
ollama.model=gemma3
```

실제 Ollama 모델명이 gemma3:4b 등이라면 ollama.model 값을 실제 설치 이름으로 변경합니다.

5. Java Spring Boot

CandleDto.java

```java
package com.wiz.upbit.dto;

import com.fasterxml.jackson.annotation.JsonProperty;

public record CandleDto(
        String market,
        @JsonProperty("candle_date_time_kst") String candleDateTimeKst,
        @JsonProperty("opening_price") double openingPrice,
        @JsonProperty("high_price") double highPrice,
        @JsonProperty("low_price") double lowPrice,
        @JsonProperty("trade_price") double tradePrice,
        @JsonProperty("candle_acc_trade_volume") double volume
) {}
```

UpbitService.java

```java
package com.wiz.upbit.service;

import com.wiz.upbit.dto.CandleDto;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;
import java.util.*;

@Service
public class UpbitService {

    private final RestClient restClient;

    public UpbitService(RestClient.Builder builder) {
        this.restClient = builder
                .baseUrl("https://api.upbit.com")
                .build();
    }

    public List<CandleDto> getXrpCandles() {
        CandleDto[] result = restClient.get()
                .uri(uriBuilder -> uriBuilder
                        .path("/v1/candles/minutes/60")
                        .queryParam("market", "KRW-XRP")
                        .queryParam("count", 200)
                        .build())
                .retrieve()
                .body(CandleDto[].class);

        if (result == null || result.length == 0) {
            throw new IllegalStateException("업비트 XRP 캔들 데이터가 없습니다.");
        }

        List<CandleDto> candles =
                new ArrayList<>(Arrays.asList(result));

        Collections.reverse(candles);
        return candles;
    }
}
```

IndicatorService.java

```java
package com.wiz.upbit.service;

import com.wiz.upbit.dto.CandleDto;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class IndicatorService {

    public double ma(List<CandleDto> candles, int period) {
        int size = candles.size();

        return candles.subList(size - period, size)
                .stream()
                .mapToDouble(CandleDto::tradePrice)
                .average()
                .orElse(0);
    }

    public double rsi(List<CandleDto> candles, int period) {
        int size = candles.size();
        double gain = 0;
        double loss = 0;

        for (int i = size - period; i < size; i++) {
            double current = candles.get(i).tradePrice();
            double previous = candles.get(i - 1).tradePrice();
            double change = current - previous;

            if (change > 0) gain += change;
            else loss += Math.abs(change);
        }

        double avgGain = gain / period;
        double avgLoss = loss / period;

        if (avgLoss == 0) return 100;

        double rs = avgGain / avgLoss;
        return 100 - (100 / (1 + rs));
    }
}
```

XrpAnalysisDto.java

```java
package com.wiz.upbit.dto;

public record XrpAnalysisDto(
        double price,
        double ma5,
        double ma20,
        double rsi,
        String aiAnalysis
) {}
```

OllamaConfig.java

```java
package com.wiz.upbit.config;

import dev.langchain4j.model.chat.ChatModel;
import dev.langchain4j.model.ollama.OllamaChatModel;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;

@Configuration
public class OllamaConfig {

    @Bean
    public ChatModel chatModel(
            @Value("${ollama.base-url}") String baseUrl,
            @Value("${ollama.model}") String modelName) {

        return OllamaChatModel.builder()
                .baseUrl(baseUrl)
                .modelName(modelName)
                .temperature(0.2)
                .build();
    }
}
```

XrpAiService.java

```java
package com.wiz.upbit.service;

import dev.langchain4j.model.chat.ChatModel;
import org.springframework.stereotype.Service;

@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String analyze(double price, double ma5,
                          double ma20, double rsi) {

        String prompt = """
            당신은 암호화폐 기술적 분석 보조 AI입니다.
            업비트 XRP 60분봉 데이터입니다.

            현재가격: %.2f
            MA5: %.2f
            MA20: %.2f
            RSI14: %.2f

            다음 형식으로 간단하게 분석하세요.
            추세:
            예상 방향: 상승 / 하락 / 횡보
            분석 이유:
            주의사항:

            확정적인 투자 조언은 하지 마세요.
            """.formatted(price, ma5, ma20, rsi);

        return chatModel.chat(prompt);
    }
}
```

XrpController.java

```java
package com.wiz.upbit.controller;

import com.wiz.upbit.dto.*;
import com.wiz.upbit.service.*;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final UpbitService upbitService;
    private final IndicatorService indicatorService;
    private final XrpAiService aiService;

    public XrpController(UpbitService upbitService,
                         IndicatorService indicatorService,
                         XrpAiService aiService) {
        this.upbitService = upbitService;
        this.indicatorService = indicatorService;
        this.aiService = aiService;
    }

    @GetMapping("/predict")
    public XrpAnalysisDto predict() {
        List<CandleDto> candles = upbitService.getXrpCandles();

        double price = candles.get(candles.size() - 1).tradePrice();
        double ma5 = indicatorService.ma(candles, 5);
        double ma20 = indicatorService.ma(candles, 20);
        double rsi = indicatorService.rsi(candles, 14);

        String ai = aiService.analyze(price, ma5, ma20, rsi);

        return new XrpAnalysisDto(price, ma5, ma20, rsi, ai);
    }
}
```

6. Kotlin Spring Boot

CandleDto.kt

```kotlin
package com.wiz.upbit.dto

import com.fasterxml.jackson.annotation.JsonProperty

data class CandleDto(
    val market: String,
    @JsonProperty("candle_date_time_kst")
    val candleDateTimeKst: String,
    @JsonProperty("opening_price")
    val openingPrice: Double,
    @JsonProperty("high_price")
    val highPrice: Double,
    @JsonProperty("low_price")
    val lowPrice: Double,
    @JsonProperty("trade_price")
    val tradePrice: Double,
    @JsonProperty("candle_acc_trade_volume")
    val volume: Double
)
```

UpbitService.kt

```kotlin
package com.wiz.upbit.service

import com.wiz.upbit.dto.CandleDto
import org.springframework.stereotype.Service
import org.springframework.web.client.RestClient

@Service
class UpbitService(builder: RestClient.Builder) {

    private val restClient = builder
        .baseUrl("https://api.upbit.com")
        .build()

    fun getXrpCandles(): List<CandleDto> {
        val result = restClient.get()
            .uri { b ->
                b.path("/v1/candles/minutes/60")
                    .queryParam("market", "KRW-XRP")
                    .queryParam("count", 200)
                    .build()
            }
            .retrieve()
            .body(Array<CandleDto>::class.java)
            ?: error("업비트 XRP 캔들 데이터가 없습니다.")

        return result.toList().reversed()
    }
}
```

IndicatorService.kt

```kotlin
package com.wiz.upbit.service

import com.wiz.upbit.dto.CandleDto
import org.springframework.stereotype.Service
import kotlin.math.abs

@Service
class IndicatorService {

    fun ma(candles: List<CandleDto>, period: Int): Double =
        candles.takeLast(period)
            .map { it.tradePrice }
            .average()

    fun rsi(candles: List<CandleDto>, period: Int): Double {
        val recent = candles.takeLast(period + 1)
        var gain = 0.0
        var loss = 0.0

        for (i in 1 until recent.size) {
            val change =
                recent[i].tradePrice - recent[i - 1].tradePrice

            if (change > 0) gain += change
            else loss += abs(change)
        }

        val avgGain = gain / period
        val avgLoss = loss / period

        if (avgLoss == 0.0) return 100.0

        val rs = avgGain / avgLoss
        return 100 - (100 / (1 + rs))
    }
}
```

XrpAnalysisDto.kt

```kotlin
package com.wiz.upbit.dto

data class XrpAnalysisDto(
    val price: Double,
    val ma5: Double,
    val ma20: Double,
    val rsi: Double,
    val aiAnalysis: String
)
```

OllamaConfig.kt

```kotlin
package com.wiz.upbit.config

import dev.langchain4j.model.chat.ChatModel
import dev.langchain4j.model.ollama.OllamaChatModel
import org.springframework.beans.factory.annotation.Value
import org.springframework.context.annotation.*

@Configuration
class OllamaConfig {

    @Bean
    fun chatModel(
        @Value("\${ollama.base-url}") baseUrl: String,
        @Value("\${ollama.model}") modelName: String
    ): ChatModel =
        OllamaChatModel.builder()
            .baseUrl(baseUrl)
            .modelName(modelName)
            .temperature(0.2)
            .build()
}
```

XrpAiService.kt

```kotlin
package com.wiz.upbit.service

import dev.langchain4j.model.chat.ChatModel
import org.springframework.stereotype.Service

@Service
class XrpAiService(
    private val chatModel: ChatModel
) {
    fun analyze(
        price: Double,
        ma5: Double,
        ma20: Double,
        rsi: Double
    ): String {

        val prompt = """
            당신은 암호화폐 기술적 분석 보조 AI입니다.
            업비트 XRP 60분봉 데이터입니다.

            현재가격: $price
            MA5: $ma5
            MA20: $ma20
            RSI14: $rsi

            다음 형식으로 분석하세요.
            추세:
            예상 방향: 상승 / 하락 / 횡보
            분석 이유:
            주의사항:

            확정적인 투자 조언은 하지 마세요.
        """.trimIndent()

        return chatModel.chat(prompt)
    }
}
```

XrpController.kt

```kotlin
package com.wiz.upbit.controller

import com.wiz.upbit.dto.XrpAnalysisDto
import com.wiz.upbit.service.*
import org.springframework.web.bind.annotation.*

@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val upbitService: UpbitService,
    private val indicatorService: IndicatorService,
    private val aiService: XrpAiService
) {
    @GetMapping("/predict")
    fun predict(): XrpAnalysisDto {
        val candles = upbitService.getXrpCandles()

        val price = candles.last().tradePrice
        val ma5 = indicatorService.ma(candles, 5)
        val ma20 = indicatorService.ma(candles, 20)
        val rsi = indicatorService.rsi(candles, 14)

        val ai = aiService.analyze(price, ma5, ma20, rsi)

        return XrpAnalysisDto(
            price = price,
            ma5 = ma5,
            ma20 = ma20,
            rsi = rsi,
            aiAnalysis = ai
        )
    }
}
```

7. Java ↔ Kotlin 핵심 비교표

작업	Java	Kotlin
DTO	record	data class
마지막 캔들	get(size - 1)	last()
최근 N개	subList()	takeLast()
역순	Collections.reverse()	reversed()
평균	Stream average()	average()
LangChain4j	동일	동일
Ollama/Gemma3	동일	동일

8. 실행 결과

Spring Boot 실행 후 다음 주소를 호출합니다.

```
http://localhost:8080/api/xrp/predict
```
text

```
예상 형태:
{
  "price": 4215.0,
  "ma5": 4198.4,
  "ma20": 4135.7,
  "rsi": 63.24,
  "aiAnalysis": "추세: 단기 상승 추세 ..."
}
```
text

9. MA와 RSI를 아주 쉽게 이해하기

MA5 = 가장 최근 5개 캔들의 종가 평균입니다. MA20 = 가장 최근 20개 캔들의 종가 평균입니다.

RSI14는 최근 가격 상승폭과 하락폭을 비교하여 0~100 범위로 나타냅니다. 입문적으로 30 이하는 과매도, 70 이상은 과매수 구간으로 해석하지만 단독 매매 신호로 사용하면 안 됩니다.

10. 중요한 설계 원칙

200개 캔들 원문 전체를 Gemma3에게 보내기보다 Spring Boot에서 MA·RSI 같은 수치를 먼저 계산하고, 계산 결과를 LLM에 전달하여 해석시키는 구조가 효율적입니다.

이 문서의 RSI는 학습용 단순 평균 방식입니다. 실전 차트와 맞추려면 다음 단계에서 Wilder RSI를 구현하고 EMA, MACD, 볼린저밴드, 거래량 지표 등을 추가할 수 있습니다.

Gemma3의 출력은 기술지표 해석 보조용이며 미래 가격이나 투자 수익을 보장하지 않습니다.
